IS507 Midterm Report

How much do body part and play pattern add to a location-based xG model?

Mengling Chu & IChen Lin

Motivation

We want to know which simple shot features make a goal-probability model more accurate, and whether its probabilities can be trusted.

The 2026 World Cup ended this summer and the European leagues are back in full swing. Football is followed by a huge audience, so it produces a lot of data. Betting sites and sports media publish win probabilities built from statistical models. We cannot build a full match-prediction model like theirs, but we can study one of its main building blocks: **expected goals (xG)**, the probability that a single shot becomes a goal.

xG turns "how good were the chances?" into a number. Analysts add it up into team totals and use it to judge players and forecast results. If the probabilities are biased, every total built on them is biased too. Building a small, transparent xG model ourselves lets us see which features really add accuracy and whether the output is reliable.

Problem

Research question: How much do body part and play pattern improve a location-based xG model, and are its probabilities trustworthy?

Goal

For every non-penalty shot, we predict the probability that it becomes a goal. This is a binary classification problem, but we care about the probability, not the yes/no label.

- **Y (target):** `is_goal` = 1 if the shot is a goal, 0 otherwise. Goals are rare: in the StatsBomb data, 3,569 of 37,488 non-penalty shots are goals, so the goal rate is 3,569 / 37,488 = 9.5%.
- **X (features):** where the shot was taken (distance and angle to goal, computed from the shot's x, y coordinates), which body part was used (foot, head, other), and how the attack started (regular play, counter-attack, corner, free kick, throw-in, other). All of these come from fields recorded on the shot event itself (`location`, `shot_body_part`, `play_pattern`). We do not use the freeze frame (the positions of the other players) yet.
- **Loss:** log loss (binary cross-entropy), the loss logistic regression minimises. It punishes confident wrong predictions heavily, so it rewards honest probabilities:

$$\text{LogLoss} = -\frac{1}{N}\sum_{i=1}^{N}\left[\, y_i \log p_i + (1-y_i)\log(1-p_i) \,\right]$$

How to read the formula. For each shot i, the model gives a goal probability p_i, and y_i is what happened (1 = goal, 0 = no goal). Only one term in the brackets is active: for a goal it is $\log p_i$, for a miss it is $\log(1 - p_i)$. So each shot is scored by the probability the model gave to what actually happened. The minus sign turns that into a penalty, and the average is taken over all N shots.

For example, predicting 90% for a shot that scores costs only $-\log 0.90 = 0.11$, but predicting 1% for a shot that scores costs $-\log 0.01 = 4.61$.

Why log loss rather than another loss.

- It scores probabilities, not yes/no answers. Accuracy only checks the yes/no answer, and a model that says "no goal" for every shot is already 90.5% accurate.
- It rewards honest probabilities: a model gets its best expected score only by reporting the true chance, so it cannot win by being over- or under-confident.
- It is the loss logistic regression minimises during training, so we train and compare models on the same measure.

We also draw a **calibration curve**: we sort shots into 10 groups by predicted probability and compare the average prediction with the actual goal rate in each group. A single number can hide where a model goes wrong; the curve shows it (for example, over-predicting the best chances). It answers the second half of our research question directly: when the model says 20%, do about 20% of those shots go in?

Data in brief

- **StatsBomb Open Data** (main data): every on-ball event of every match, including each shot's location, body part, play pattern and outcome. We use the 2015/16 season because it is the one season where StatsBomb's free data covers complete seasons of several top leagues; most other free seasons follow a single team. Shot-level labels of this quality are normally paid data.
- **football-data.co.uk** (supporting data): one row per match with results and pre-match bookmaker odds. The site collects pre-match odds from several bookmakers; Bet365 is one large bookmaker, and its columns `B365H`, `B365D`, `B365A` give the odds for a home win, draw and away win. We use one bookmaker so that every match is measured the same way. The result is each team's pre-match win probability, a measure of team strength. This lets us check whether our probabilities are equally trustworthy for strong and weak teams.

Potential impact

- Shows which cheap, easy-to-record features are worth adding to a location-only model. That matters for leagues or clubs that have event data but no player-tracking data.
- xG is usually added up: a team's xG for a season is the sum of its shots' probabilities, read as "how many goals it should have scored". This total is only fair if the probabilities are well calibrated. Our calibration checks show whether such totals can be used as they are, without correction.
- Keeps the model interpretable: every coefficient can be read as an odds ratio, such as how much a header changes the odds of scoring compared with a foot shot from the same spot.

Related work

xG modelling is well established. Two pieces of work shape our design.

- **Robberechts & Davis (2020), KU Leuven.** They compared xG models trained on different amounts and kinds of data. A logistic regression with basic features (location, distance, angle, body part) stops improving after about 6,000 training shots, roughly two-thirds of a season. They argue that the main goal of an xG model is calibrated probabilities, checked with calibration curves. For us, this means our 26,201 training shots are enough for a simple model, and calibration belongs at the centre of our evaluation.
- **StatsBomb xG** (included in our data as `shot_statsbomb_xg`). A commercial model that also uses the positions of the other players at the moment of the shot. We use it as an expert reference line, never as a feature. The gap between our model and StatsBomb's shows how much the information we leave out is worth.

Data

We model 37,488 non-penalty shots from 1,517 matches in four complete 2015/16 leagues (Premier League, La Liga, Serie A, Ligue 1). The Bundesliga is also in the source but is excluded because its data is incomplete (see Challenges below). The shot table is built by us from StatsBomb's raw event data.

How the sources are organised

StatsBomb Open Data is a set of linked JSON files, not one table. A `competitions.json` file lists every available league and season, and four folders hold the rest, linked by `match_id`:

- **Used:** `matches/` (one file per competition-season, one record per match) and `events/` (one file per match, one record per on-ball action, including every shot; the source of our shot table).
- **Not used:** `lineups/` (squads per match) and `three-sixty/` (positions of all visible players at each event).

We use `matches/` and `events/` for the 2015/16 season: we pick the four leagues from `competitions.json`, list their matches from `matches/`, then download each match's events and keep only the shots.

football-data.co.uk is simpler: one CSV file per league and season, one row per match.

What the data look like

Dataset	Source	One row =	Rows	Columns	Column types
Raw events	StatsBomb Open Data, JSON via `statsbombpy`	one on-ball action (pass, carry, shot, …)	5,321,459 (mean 3,508 per match)	120	text/category, boolean, numeric, nested lists/dicts
Match list	StatsBomb	one match	1,517	55	36 text/category, 19 numeric
Shot table (modelling data)	built by us from raw events	one shot	37,888 (37,488 after removing 400 penalties)	20	11 text/category, 7 numeric, 2 boolean
Match odds	football-data.co.uk, CSV	one match (result, pre-match odds)	1,520 (380 per league)	65 (we use 6: date, home team, away team, three Bet365 odds)	58 numeric (about 35 are bookmaker odds), 7 text (league, date, teams, results, referee)

Shots are only 0.7% of all events: passes (27.8%), ball receipts (26.0%) and carries (21.0%) dominate. The data mix several types: categories (body part, play pattern), numbers (x, y coordinates on a 120 × 80 pitch), true/false flags, and nested lists (the freeze frame: every player's position at the moment of the shot).

Shot-table columns we use:

Column	Type	Example	Role
`x`, `y`	numeric	100.0, 27.9	Feature: distance and angle are computed from these
`shot_body_part`	category	Right Foot	Feature (Model 2)
`play_pattern`	category	From Counter	Feature (Model 3)
`shot_outcome`	category	Saved	Target: Goal = 1, everything else = 0
`shot_type`	category	Open Play	Used to remove penalties
`match_id`	numeric	3754173	Used to split train/validation/test

Column	Type	Example	Role
shot_statsbomb_xg	numeric	0.040	Reference only, never a feature

Challenges we already found

- **Raw data is nested JSON, not a table:** one events file per match; locations and freeze frames are nested lists → flattened shots into our own table.
- **Bundesliga is incomplete:** only 34 of 306 matches, all involving Leverkusen; the source file itself holds only these 34 → excluded from modelling.
- **Ligue 1 is missing 3 matches:** 377 of 380; checked against football-data, no common pattern → kept as a random coverage gap.
- **Penalties behave differently:** 400 penalties, 75.0% scored vs 9.5% for other shots → removed from modelling.
- **"Missing" values that are not missing:** `under_pressure` 74.1% blank, `shot_first_time` 68.7% blank, because StatsBomb records these only when true → found during cleaning and recorded the rule (blank = False); neither column is a feature in our current models.
- **Class imbalance:** only 9.5% of shots are goals → use log loss and calibration, not accuracy.
- **Shots are not independent:** 25 shots per match; the top 10% of shooters take 36% of shots → split by match, never by shot.
- **Joining two sources:** 45 of 98 team names differ (e.g. "Man United") and date formats are mixed → manual name map and mixed-format date parsing.

Challenges we expect

- **Confounding:** headers come mostly from crosses and corners close to goal, so raw goal rates by body part can mislead. Only a model that holds location fixed can separate the effects.
- **Small groups:** some categories (shots with "other" body parts, rare play patterns) have few shots, so their estimates will be noisy.
- **Missing information:** we do not use defender or goalkeeper positions, so we expect a gap to StatsBomb's own xG.
- **One season only:** results may not carry over to other seasons or leagues.

Plan

We fit a series of logistic regressions. Each one adds a single group of features, so every improvement can be traced to one idea.

Models

1. **Baseline 0:** predict the training goal rate (9.3%) for every shot.
2. **Baseline 1:** distance + angle (location only).
3. **Model 2:** + body part (header, other; foot is the reference).
4. **Model 3:** + play pattern (counter, corner, free kick, throw-in, other; regular play is the reference).

For the midterm, we have built and evaluated Baselines 0 and 1. Models 2 and 3, the odds check, and the test-set evaluation are planned for the final.

Why logistic regression: the output is a probability between 0 and 1, it is the standard xG starting point, and each coefficient converts to an odds ratio we can explain.

Each model turns a shot's features into a goal probability:

$$p = \frac{1}{1 + e^{-(b_0 + b_1 x_1 + b_2 x_2 + \cdots + b_k x_k)}}$$

Here the *x* values are the features (distance, angle, body part, …) and the *b* values are the coefficients. Training means finding the coefficients that make the log loss on the training set as small as possible.

Training and evaluation

We split by **match** (randomly), so all shots from one match stay in the same set: train 70% (fitting models, error analysis), validation 15% (comparing models), test 15% (sealed until the final).

- Fit each model on the training set by minimising log loss (scikit-learn `LogisticRegression`).
- Compare models on the validation set with log loss and calibration curves, with StatsBomb's xG as a reference line.
- Error analysis on the training set: compare actual goals with predicted goals by body part, play pattern, league and distance band. A z-score above 2 flags a real bias rather than chance.
- Next step: join the football-data odds and repeat the check by pre-match team strength, to see whether the probabilities hold for favourites and underdogs alike.
- Report the sealed test set once, for the chosen model only.

Baselines and why

- **Baseline 0** is the floor. It knows nothing about the shot, so any model that uses shot information must beat it; if not, the features add nothing.
- **Baseline 1** is the classic location-only xG model and the starting point of our research question. Every later model is judged by how much it improves on Baseline 1. Fitted on the training set: intercept −1.527, distance −0.081 per yard, angle +1.146 per radian. Shots further away score less often; shots with a wider view of the goal score more often.
- An error analysis of Baseline 1 on the training set shows why the next features matter: it predicts 849 goals from headers but only 451 were scored, over-predicts goals from corners, and under-predicts goals from counter-attacks.